Plataforma de Riesgo Crediticio de Extremo a Extremo sobre Lending Club: Documentación del Proyecto en Formato de Entrevista Técnica

Wilder Espinoza Luna
Escuela Profesional de Estadística, Facultad de Ciencias Matemáticas, Universidad Nacional Mayor de San Marcos

Nota del autor

Este documento acompaña al repositorio *Credit Risk Platform 2.0* (https://github.com/wilder14-eslu/credit-risk-platform-2.0) y a su demo pública (https://credit-risk02.streamlit.app). Está escrito para preparar entrevistas técnicas de Machine Learning Engineer y Data Scientist.

Las cifras provienen del reporte reproducible del proyecto (`reports/results.json`, generado el 4 de octubre de 2026) y de la corrida de producción simulada en Databricks de esa misma fecha. Versión del documento: 12 de octubre de 2026.

Resumen

Se documenta, en formato de entrevista técnica, una plataforma de riesgo crediticio que estima la probabilidad de default (PD) de solicitudes de préstamo y decide su aprobación. El sistema se construyó sobre 2,260,701 préstamos reales de Lending Club (2007-2018) y se desplegó por completo en Databricks Free Edition con Delta Lake, Unity Catalog, MLflow, Model Serving y Databricks Apps, orquestado como código con Asset Bundles y GitHub Actions. La validación es temporal: entrenamiento con préstamos de 2007 a 2012, selección en el primer semestre de 2013 y confirmación única en el segundo semestre de 2013. Cuatro algoritmos y un scorecard WoE se compararon con el test de DeLong, el ajuste de Holm, un margen práctico de 0.005 de AUC y una regla de parsimonia. El modelo elegido, CatBoost ajustado con Optuna, obtuvo un AUC fuera de tiempo de 0.6912, IC 95 % [0.6863, 0.6960], y un KS de 0.277. Con un umbral de 0.18, elegido por costo esperado con una relación 5:1, el modelo aprueba el 61.3 % de las solicitudes y reduce la morosidad de los aprobados de 15.5 % a 9.6 %. Un replay mes a mes de la originación de 2014-2015 mostró concept drift y data drift reales, disparó reentrenamientos automáticos y un A/B champion-challenger que, sin evidencia significativa (p = .61), mantuvo al champion. Se discuten las limitaciones de calibración, de disponibilidad de la etiqueta y de potencia estadística del A/B.

Palabras clave: riesgo crediticio, probabilidad de default, MLOps, Databricks, test de DeLong, concept drift, A/B testing

Plataforma de Riesgo Crediticio de Extremo a Extremo sobre Lending Club: Documentación del Proyecto en Formato de Entrevista Técnica

Este documento presenta el proyecto como una entrevista técnica. Cada encabezado de tercer nivel es una pregunta que suele hacer un entrevistador, y el texto que le sigue es la respuesta del candidato, en primera persona. La estructura sigue el estilo APA 7: la introducción plantea el problema y el alcance, el método describe los datos, el modelado y la plataforma, los resultados reportan el desempeño fuera de tiempo y en producción simulada, y la discusión cubre decisiones, limitaciones y preguntas difíciles. El Apéndice A reúne las cifras que conviene memorizar y el Apéndice B, preguntas rápidas de seguimiento.

Las respuestas se apoyan en lo que está implementado y medido en el repositorio (Espinoza Luna, 2026). Las respuestas de la discusión que proyectan mejoras o lecciones son propuestas para adaptar con palabras propias.

Presentación del Proyecto

P1. ¿Puedes presentarme el proyecto en dos minutos?

Construí una plataforma de riesgo crediticio de extremo a extremo que estima la probabilidad de default de una solicitud y decide si aprobarla. Usé 2.26 millones de préstamos reales de Lending Club, emitidos entre 2007 y 2018, y desplegué todo en Databricks Free Edition: ingesta en Delta Lake con arquitectura medallion, una feature table en Unity Catalog, entrenamiento con cuatro algoritmos, registro en MLflow, Model Serving con dos variantes, una API en FastAPI y un dashboard en Streamlit.

Lo que diferencia al proyecto no es el modelo, sino el ciclo de vida. Elegí el modelo con validación temporal y pruebas estadísticas, y después lo enfrenté mes a mes a préstamos de 2014 y 2015 que nunca había visto. El monitoreo detectó concept drift y data drift reales, disparó reentrenamientos y un A/B champion-challenger decidió no reemplazar al modelo vigente porque no había evidencia de que el nuevo fuera mejor. Todo se despliega como código, con entornos dev, staging y prod, 100 tests y una cobertura cercana al 94 %.

En números: el AUC fuera de tiempo es 0.69 y, con el umbral elegido por costo, la morosidad de la cartera aprobada baja de 15.5 % a 9.6 % aprobando al 61 % de los solicitantes.

P2. ¿Qué problema de negocio resuelve?

Ante cada solicitud, el prestamista debe aprobar o rechazar. La plataforma entrega cuatro cosas: la probabilidad de default, la decisión (APROBAR o RECHAZAR), una banda de riesgo de la A a la E y las tres variables que más empujaron la predicción. Las dos últimas hacen que la decisión sea explicable ante el solicitante y ante un regulador.

El costo de equivocarse no es simétrico. Otorgar un préstamo a quien incumple cuesta más que rechazar a un buen pagador, así que lo modelé con una relación de costos de 5 a 1 y elegí el umbral que minimiza el costo esperado, en lugar de usar 0.5 por defecto.

P3. ¿Por qué elegiste Lending Club y Databricks?

Lending Club tiene préstamos reales con su desenlace y cubre más de una década, lo que permite validar en el tiempo y observar drift genuino sin inyectarlo de forma artificial (wordsforthewise, s. f.). Además es un dataset con trampas conocidas, como columnas de leakage y sesgo de censura, que obligan a tomar decisiones de diseño como en un banco.

Elegí Databricks porque concentra en una sola plataforma lo que un equipo de ML usa en producción: Delta Lake, Unity Catalog, MLflow, Model Serving, Jobs y Apps (Sinha, 2023). Con la Free Edition diseñé dentro de restricciones reales: solo compute serverless, cinco tareas concurrentes, un SQL warehouse 2X-Small y apps que se apagan a las 24 horas (Databricks, s. f.). Esta es la versión 2.0 del proyecto; la primera usaba Docker, PostgreSQL y Prefect con otro dataset, y la reconstruí desde cero sobre Databricks.

P4. ¿Qué alcance tiene y qué fases del ciclo de vida cubre?

Cubre el ciclo completo: planteamiento, datos, calidad, variables, validación, modelado, explicabilidad, despliegue, monitoreo, experimentación, reentrenamiento y operación. La Tabla 1 resume las doce fases y dónde vive cada una en el repositorio.

Tabla 1

Fases del Ciclo de Vida Cubiertas por la Plataforma

Fase	Qué se hace	Dónde
1. Planteamiento	Decisión de negocio, target y costo asimétrico 5:1	`docs/REPORTE_TECNICO.md`
2. Datos	Parser del CSV crudo, 22 columnas de leakage eliminadas, control de censura	`src/credit_risk/data/`
3. Calidad	Expectativas bloqueantes en `data_quality_log`	`data/quality.py`
4. Variables	26 de la solicitud, 5 derivadas e indicadores de faltantes	`src/credit_risk/features/`
5. Validación	Split temporal: 2007-2012, 2013-H1 y 2013-H2	`models/training.py`
6. Modelado	Benchmark, Optuna, umbral por costo, quality gates	`jobs/03` y `jobs/04`
7. Explicabilidad	SHAP por solicitud (tres razones) y bandas A a E	`models/credit_model.py`
8. Despliegue	Registro en UC, Model Serving con A/B, API y dashboard	`registry/` y `apps/`
9. Monitoreo	Data, prediction y concept drift; prior shift	`monitoring/`, `jobs/06`
10. Experimentación	A/B champion-challenger, promoción y rollback	`jobs/07` y `jobs/08`
11. Reentrenamiento	CT semanal y disparado por drift, con ventanas desplazadas	`resources/jobs.yml`
12. Operación	Reintentos idempotentes, timeouts, alertas, lineage, entornos	`docs/DEPLOYMENT.md`

Nota. Las rutas son relativas a la raíz del repositorio o a `src/credit_risk/`.

Método

Datos y Definición del Problema

P5. ¿Cómo definiste la variable objetivo?

La variable `target_default` vale 1 si el estado final del préstamo es *Charged Off* o *Default* y 0 si es *Fully Paid*; también mapeé las variantes que Lending Club marca como fuera de su política de crédito. Los préstamos *Current*, *Late* o *In Grace Period* no tienen desenlace, así que no se etiquetan. Como predictores solo uso información disponible al momento de la solicitud.

P6. ¿Cómo evitaste el data leakage?

Es el error más común con este dataset. Identifiqué 22 columnas que solo existen después del desembolso, como `total_pymnt`, `recoveries`, `last_fico_range_high` o `debt_settlement_flag`. Las elimino en la capa Silver, y una expectativa bloqueante, `no_leakage_columns`, detiene el pipeline si alguna reaparece; además, un test lo verifica en CI. Mi regla práctica es que, en Lending Club, un AUC de 0.90 o más con variables de solicitud es casi siempre una señal de leakage y no de un buen modelo.

P7. ¿Qué es el sesgo de censura por la derecha y cómo lo trataste?

Entre los préstamos recientes, los que ya tienen desenlace son sobre todo los que incumplieron temprano. Si los etiqueto, el modelo aprende un artefacto: casi todos los préstamos a 60 meses de 2016 con desenlace son defaults, y el modelo concluiría que el plazo de 60 meses equivale a default. La Tabla 2 muestra cómo cae la proporción de préstamos con desenlace conocido en los años recientes.

La solución fue etiquetar un préstamo solo si su plazo terminó antes del corte del archivo (`label_snapshot` = 2019-01-01). Por eso el entrenamiento usa 2007-2012, la validación y el test usan 2013, y el replay de producción usa 2014-2015, cuyos préstamos a 36 meses ya vencieron.

Tabla 2

Préstamos por Año de Emisión y Proporción con Desenlace Conocido

Año de emisión	Préstamos	Con desenlace conocido
2007-2012	95,902	100 %
2013	134,814	~100 %
2014	235,629	95 %
2015	421,095	89 %
2016	434,407	67 %
2017	443,579	38 %
2018	495,242	11 %

Nota. Medido sobre el archivo completo `accepted_2007_to_2018Q4.csv` (2,260,701 préstamos), antes del filtro de madurez.

P8. ¿Cómo controlas la calidad de los datos?

En cada corrida evalúo expectativas y las registro en la tabla `data_quality_log`: número mínimo de filas, unicidad del identificador, fechas válidas, tasas máximas de nulos, categorías conocidas, una tasa de default entre 3 % y 40 % y la ausencia de columnas de leakage. Las de severidad *error* detienen el pipeline, de modo que un archivo corrupto nunca llega al entrenamiento.

Variables

P9. ¿Qué variables usaste y cuáles construiste?

Uso 26 variables de la solicitud. Hay 18 numéricas: monto, plazo, tasa, cuota, antigüedad laboral, ingreso, DTI, FICO, morosidades, consultas, líneas de crédito, saldo y utilización revolvente, hipotecas, quiebras y antigüedad crediticia. Hay 8 categóricas: grado, subgrado, vivienda, verificación de ingresos, propósito, estado, tipo de solicitud y listado inicial. Construí 5 derivadas: monto sobre ingreso, cuota anual sobre ingreso, saldo revolvente sobre ingreso, líneas abiertas sobre totales y el logaritmo del ingreso.

Para los faltantes agrego indicadores binarios antes de imputar con la mediana, porque en crédito el dato faltante es informativo: un `emp_length` vacío suele indicar que la persona no declaró empleo. Las categorías con menos de 0.5 % de frecuencia se agrupan para que el modelo no memorice niveles raros.

P10. ¿Cómo evitas el training-serving skew?

El preprocesador se ajusta solo con el periodo de entrenamiento y viaja serializado dentro del modelo registrado en MLflow. Así el endpoint aplica exactamente la misma transformación que el entrenamiento y el scoring batch, sin depender de online tables, que la Free Edition no ofrece.

Diseño de Validación

P11. ¿Por qué un split temporal y no uno aleatorio?

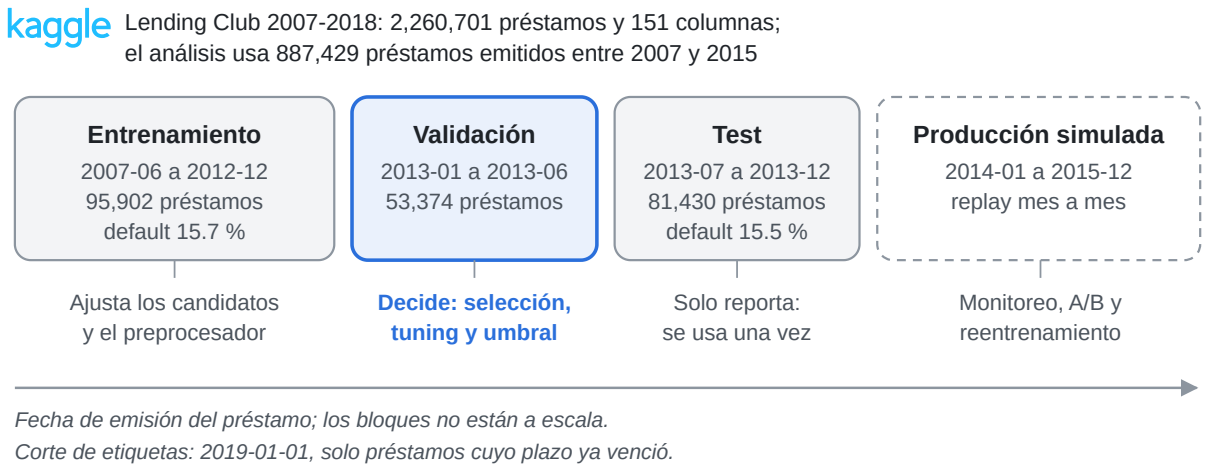
Porque así se valida un scorecard en banca: el modelo se mide en préstamos posteriores a los que vio (Siddiqi, 2017). Un split aleatorio mezcla cosechas y sobreestima el desempeño, porque el modelo ve el futuro. La Figura 1 muestra el diseño: entrenamiento con

95,902 préstamos de 2007 a 2012, validación con 53,374 del primer semestre de 2013, test con 81,430 del segundo semestre de 2013 y producción simulada de 2014 a 2015.

Cuando el monitoreo dispara un reentrenamiento, las ventanas se desplazan con la fecha del replay (`as_of`): el test pasa a ser los últimos seis meses con desenlace observado, la validación los seis meses previos y el entrenamiento todo lo anterior.

Figura 1

Diseño de Validación Temporal



Nota. La validación decide y el test se usa una sola vez. El logotipo de Kaggle identifica la fuente de los datos (wordsforthewise, s. f.).

P12. ¿Usas el test para elegir el modelo?

No. La validación decide todo: la selección del algoritmo, el tuning y el umbral. El test se usa una sola vez para reportar y para los quality gates del modelo ya elegido. Si eligiera el mejor de varios modelos sobre el test, el AUC reportado tendría un sesgo optimista, la llamada maldición del ganador.

Modelado y Selección Estadística

P13. ¿Qué modelos comparaste y cómo los ajustaste?

Comparé regresión logística (C = 0.5), XGBoost (Chen y Guestrin, 2016), LightGBM (Ke et al., 2017) y CatBoost (Prokhorenkova et al., 2018) con hiperparámetros conservadores: árboles poco profundos, regularización L2 y un mínimo alto de muestras por hoja. El algoritmo ganador se ajusta con Optuna (Akiba et al., 2019), con hasta 20 intentos o 15

minutos. El objetivo es el AUC de validación penalizado por sobreajuste: si la brecha entre entrenamiento y validación supera 0.08, el exceso se resta del puntaje. Si el modelo ajustado no pasa los gates, recurro al mejor candidato con parámetros por defecto que sí los pase.

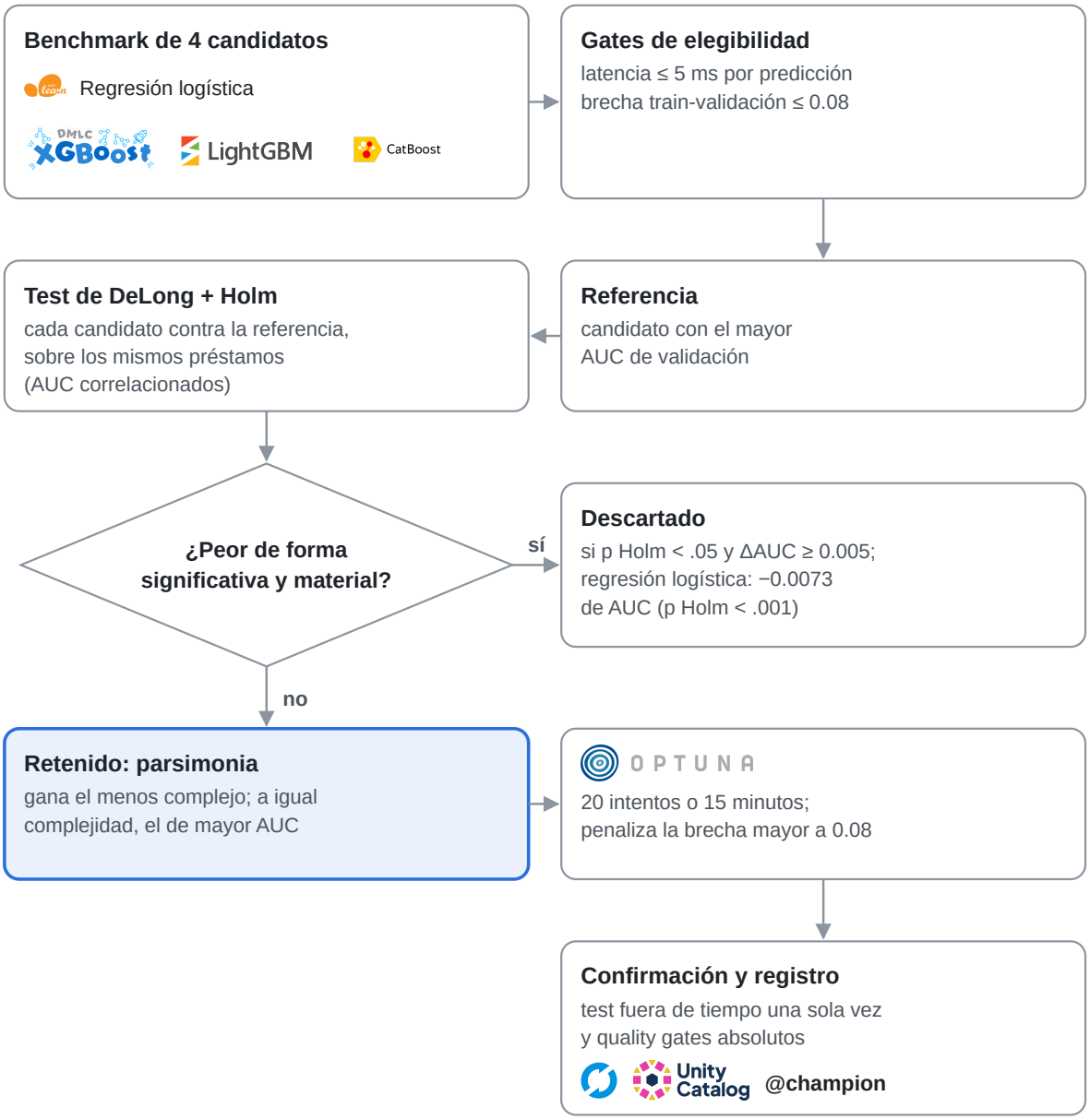
P14. ¿Cómo eliges el modelo ganador?

Con una regla estadística (Figura 2). Primero filtro por latencia y brecha de sobreajuste. Luego tomo como referencia al candidato de mayor AUC de validación y comparo cada otro contra ella con el test de DeLong para AUC correlacionados (DeLong et al., 1988; Sun y Xu, 2014), porque todos se evalúan sobre los mismos préstamos; ajusto los *p*-valores con el método de Holm (1979) por comparaciones múltiples. Descarto un candidato solo si es peor de forma significativa (p Holm $< .05$) y material ($\Delta\text{AUC} \geq 0.005$). Entre los que sobreviven gana el menos complejo y, a igual complejidad, el de mayor AUC.

Así no premio con más complejidad una diferencia que es ruido. Con decenas de miles de préstamos, diferencias muy pequeñas resultan significativas; el margen práctico separa lo estadísticamente detectable de lo relevante para el negocio.

Figura 2

Regla de Selección Estadística del Modelo



Nota. ΔAUC = diferencia de AUC contra la referencia en la validación. Los logotipos pertenecen a sus respectivos titulares y se usan solo para identificar cada tecnología.

P15. ¿Por qué incluiste un scorecard WoE si no lo despliegas?

Porque es la referencia que entiende y exige un regulador (Siddiqi, 2017). Implementé binning por cuantiles, Weight of Evidence con suavizado, Information Value, una regresión logística sobre WoE y el escalado a puntos, con PDO de 20 y 600 puntos para odds de 50 a 1.

El scorecard responde una pregunta clave: si la complejidad de un GBM se justifica frente al estándar de la industria. En este caso sí, como muestran los resultados.

P16. ¿Cómo fijaste el umbral de decisión?

Minimizando el costo esperado 5·FN + 1·FP en la validación. El umbral resultante fue 0.18: se rechaza toda solicitud con PD mayor a 18 %. Fijar 0.5 habría optimizado la exactitud y no el negocio; con una prevalencia de 15.5 %, ese umbral habría aprobado a casi todos los solicitantes.

P17. ¿Qué controles debe pasar un modelo antes de registrarse?

Hay gates absolutos: AUC de test ≥ 0.66, brecha de sobreajuste ≤ 0.08, Brier ≤ 0.20 y latencia ≤ 5 ms por predicción; si alguno falla, no se registra nada. Además hay un gate relativo: un challenger solo reemplaza al champion si mejora el AUC en al menos 0.002, evaluando a ambos sobre el mismo test fuera de tiempo.

P18. ¿Cómo explicas una predicción individual?

Con contribuciones SHAP por solicitud (Lundberg y Lee, 2017). Sumo las contribuciones de las variables dummy de una misma categórica para hablar de la variable original y devuelvo las tres que más empujaron la predicción. También asigno una banda de riesgo: A (PD < 5 %), B (< 10 %), C (< 20 %), D (< 35 %) y E. Eso permite explicar un rechazo en lenguaje de negocio.

Arquitectura y MLOps

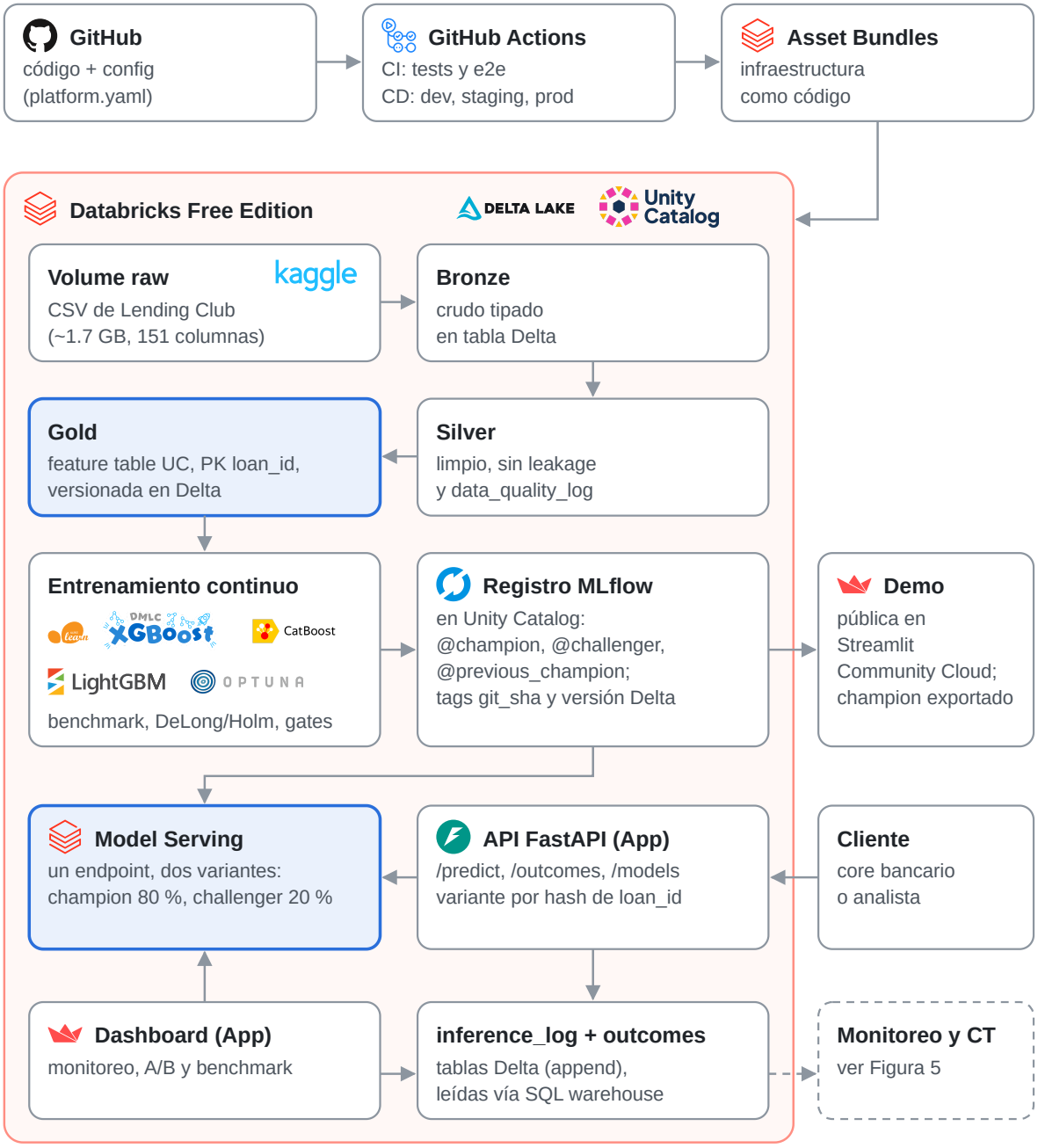
P19. ¿Cómo es la arquitectura de la plataforma?

Todo vive en Databricks (Figura 3). El CSV crudo entra a un Volume; de ahí pasa a Bronze (crudo tipado), Silver (limpio y validado) y Gold, una feature table en Unity Catalog con clave primaria `loan_id`, versionada en Delta. El pipeline de entrenamiento lee Gold, compara candidatos y registra el ganador en MLflow con los aliases `@champion`, `@challenger` y `@previous_champion`. Un endpoint de Model Serving sirve dos variantes, y la API en FastAPI y el dashboard en Streamlit corren como Databricks Apps. La infraestructura se declara con Asset Bundles y se despliega desde GitHub Actions.

Como las Apps de la Free Edition exigen el login del workspace y se apagan a las 24 horas, publiqué una demo autocontenida en Streamlit Community Cloud con el champion exportado de Unity Catalog y una foto de las tablas de producción.

Figura 3

Arquitectura de la Plataforma en Databricks Free Edition



Nota. Las cajas azules marcan la feature table y el endpoint de serving, los dos puntos que comparten entrenamiento y producción. Los logotipos pertenecen a sus respectivos titulares y se usan solo para identificar cada tecnología.

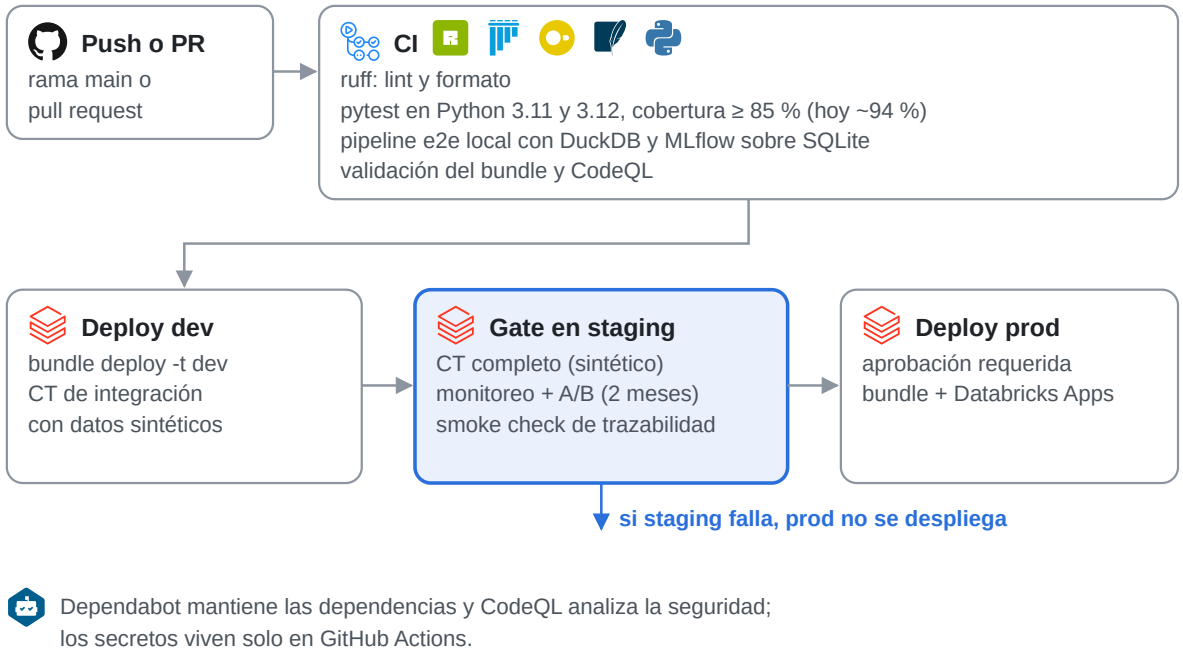
P20. ¿Cómo funciona tu CI/CD?

En cada push o pull request, CI ejecuta ruff, 100 tests en Python 3.11 y 3.12 con una cobertura mínima de 85 % (hoy cerca de 94 %), un pipeline end-to-end que corre todos los jobs en local con DuckDB y MLflow sobre SQLite, la validación del bundle y CodeQL (Figura 4). Después, CD despliega en dev, ejecuta un gate de integración en staging (entrenamiento completo con datos sintéticos, monitoreo con A/B y un smoke check de trazabilidad) y solo entonces despliega en prod, con aprobación manual.

Sigo el patrón *deploy code*: despliego el código y el modelo se entrena en cada entorno (Sinha, 2023). Como la Free Edition tiene un solo workspace, aíslo los entornos por esquema de Unity Catalog: `credit_risk_dev`, `credit_risk_staging` y `credit_risk`.

Figura 4

Flujo de Integración y Despliegue Continuos



Nota. Staging no despliega el endpoint de Model Serving para ahorrar cuota; el endpoint se valida en dev. Los logotipos pertenecen a sus respectivos titulares y se usan solo para identificar cada tecnología.

P21. ¿Cómo garantizas la trazabilidad y la reproducibilidad?

Cada versión del modelo guarda como tags su commit (`git_sha`), la tabla y la versión Delta exacta de los datos de entrenamiento (`gold_delta_version`) y los cortes temporales. Con *time travel* puedo reconstruir los datos de cualquier modelo en producción mediante `SELECT * FROM <tabla_gold> VERSION AS OF <versión>`. En CD, un smoke check exige que el `@champion` tenga toda esa trazabilidad.

P22. ¿Cómo sirves el modelo y asignas el tráfico del A/B?

Uso un solo endpoint de Model Serving con dos variantes: el champion recibe el 80 % del tráfico y el challenger el 20 %. La API asigna la variante con un hash SHA-256 del `loan_id`, así que un mismo solicitante siempre ve la misma variante y la asignación es reproducible en el replay. Si el challenger falla, la API responde con el champion para no cortar el servicio. Cada predicción se registra en Delta con su variante, la versión del modelo y la latencia.

P23. ¿Cómo simulaste la producción?

En lugar de inyectar drift artificial, re-jugué la originación real de 2014 y 2015 mes a mes, en el orden en que ocurrió. Cada corrida avanza tres meses de reloj simulado, con hasta 4,000 préstamos por mes para cuidar la cuota de compute, y las etiquetas llegan con seis meses de retraso, como en un banco. Así el monitoreo enfrenta los cambios que de verdad tuvo el portafolio. Los datos y el drift son reales; el entorno de producción es una simulación.

P24. ¿Qué tipos de drift monitoreas?

Separo cuatro tipos según qué distribución cambia (Tabla 3). Como el desenlace llega tarde, el data drift funciona como alerta temprana y el concept drift se confirma cuando llegan las etiquetas. Para el concept drift secuencial uso DDM sobre los errores (Gama et al., 2004) y Page-Hinkley sobre la log-loss (Page, 1954). Los umbrales están versionados en `config/platform.yaml`, así que cambiarlos pasa por CI/CD.

Tabla 3

Tipos de Drift Monitoreados, Método de Detección y Umbral de Alerta

Tipo	Qué cambia	Detección	Umbral de alerta
Data drift	$P(X)$	PSI por variable y test KS	PSI ≥ 0.25 en 3 o más variables
Prediction drift	$P(\hat{y})$	PSI de los scores	PSI ≥ 0.20
Concept drift	$P(y \mid X)$	Caída de AUC y KS; aumento del Brier	0.04, 0.08 y 0.02
Concept drift secuencial	$P(y \mid X)$	DDM sobre errores; Page-Hinkley sobre log-loss	2σ aviso y 3σ drift (DDM)
Prior shift	$P(y)$	Test *z* de dos proporciones	Cambio absoluto ≥ 3 puntos

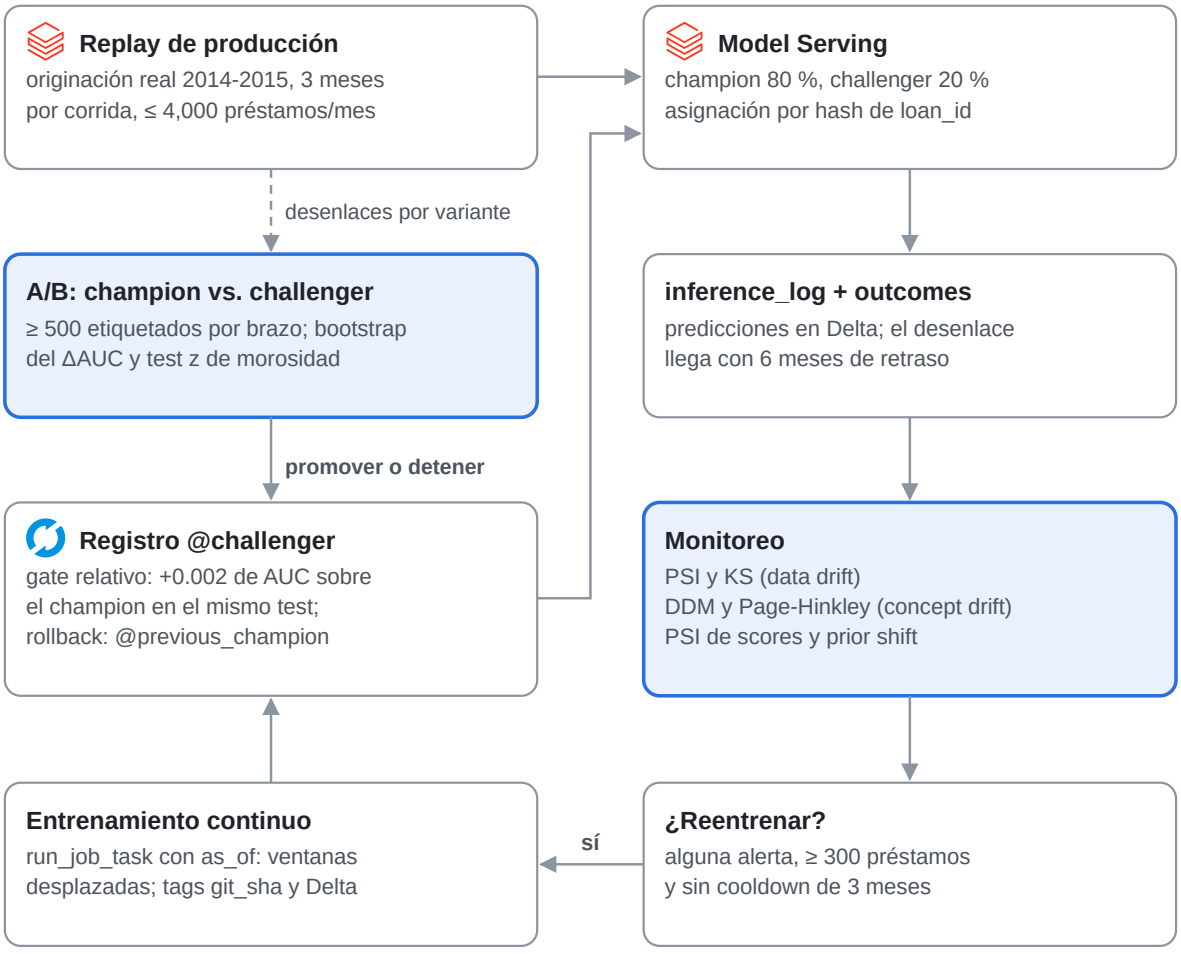
Nota. PSI = índice de estabilidad poblacional; KS = Kolmogorov-Smirnov; DDM = Drift Detection Method.

P25. ¿Cuándo se dispara el reentrenamiento?

Cualquier señal de alerta dispara Continuous Training, salvo que la ventana tenga menos de 300 préstamos o que haya habido un reentrenamiento en los últimos tres meses de reloj simulado. El job de monitoreo invoca al pipeline de entrenamiento con `run_job_task` y le pasa la fecha del replay para desplazar las ventanas; además hay un schedule semanal. La Figura 5 muestra el ciclo completo.

Figura 5

Ciclo Cerrado de Monitoreo, A/B Testing y Reentrenamiento



Nota. El job de monitoreo ejecuta en secuencia el replay, la detección de drift, la evaluación del A/B y, si corresponde, el disparo del entrenamiento continuo. Los logotipos pertenecen a sus respectivos titulares y se usan solo para identificar cada tecnología.

P26. ¿Cómo decides si el challenger reemplaza al champion?

Cuando cada brazo tiene al menos 500 préstamos con desenlace, comparo con un bootstrap pareado de la diferencia de AUC (500 réplicas, $\alpha = .05$) y con un test z de dos proporciones sobre la morosidad de los aprobados. Promuevo si el challenger es significativamente mejor y no daña al negocio; lo detengo si es peor, si aprueba más morosos o si se cumplen seis meses sin diferencia; en otro caso el test continúa. Un job de rollback devuelve `@champion` a `@previous_champion` si algo sale mal.

P27. ¿Qué nivel de madurez MLOps alcanza y cómo cuidaste la confiabilidad?

Alcanza el nivel 2 del marco de Google Cloud (s. f.): pipeline de entrenamiento automatizado, validación de datos y de modelo, feature store y metadata, entrenamiento continuo por schedule y por drift, CI con tests y pipeline end-to-end, CD del pipeline y del modelo por entornos, y monitoreo con experimentación online, promoción automática y rollback.

En operación, cada job tiene timeouts, cola y alertas por fallo y por duración. Solo reintento tareas idempotentes: la ingesta y las capas Silver y Gold sobrescriben tablas, y el despliegue del endpoint es declarativo. No reintento el entrenamiento (un gate fallido no mejora repitiendo), ni el replay, el monitoreo y el A/B (hacen append), ni el rollback. Un test de contrato hace cumplir esas reglas.

Resultados

Desempeño Fuera de Tiempo

P28. ¿Qué resultados obtuvo el benchmark?

Los cuatro algoritmos quedaron en un rango de 0.007 de AUC en el test (Tabla 4). Ese hallazgo es un resultado en sí mismo: el límite lo pone la información disponible en la solicitud y no el algoritmo. La regresión logística tiene la menor brecha de sobreajuste, pero también el menor AUC; los GBM generalizan con brechas de 0.03 a 0.04 y latencias menores a 0.1 ms por predicción.

Tabla 4

Benchmark de Candidatos con Parámetros por Defecto

Modelo	AUC train	AUC val.	AUC test	Brecha train-val.	KS test	Brier test	Latencia (ms)
CatBoost	0.7224	0.6932	0.6901	+0.0292	0.2744	0.1235	0.076
LightGBM	0.7363	0.6925	0.6896	+0.0437	0.2745	0.1236	0.078
XGBoost	0.7266	0.6917	0.6892	+0.0349	0.2732	0.1235	0.088
Regresión logística	0.6975	0.6860	0.6835	+0.0115	0.2682	0.1244	0.077

Nota. Validación: préstamos emitidos de enero a junio de 2013. Test: de julio a diciembre de 2013. Latencia por predicción.

P29. ¿Por qué ganó CatBoost si las diferencias son tan pequeñas?

Porque la regla lo hace explícito (Tabla 5). CatBoost tuvo el mayor AUC de validación (0.6932). LightGBM y XGBoost quedaron a menos de 0.002 y se retienen por empate práctico; la diferencia con XGBoost es estadísticamente detectable (p Holm = .011), pero no material. La regresión logística queda descartada: es peor por 0.0073, con p Holm < .001. Entre los tres GBM, con igual complejidad, gana el de mayor AUC. Para el negocio, la elección entre ellos es indiferente.

El test confirma sin decidir (Figura 6). Frente a la regresión logística, el modelo final gana 0.0066 de AUC, IC 95 % [0.0052, 0.0081], y frente al scorecard WoE gana 0.0089, IC 95 % [0.0073, 0.0105]; ambas diferencias son significativas y materiales tras el ajuste de Holm.

Tabla 5

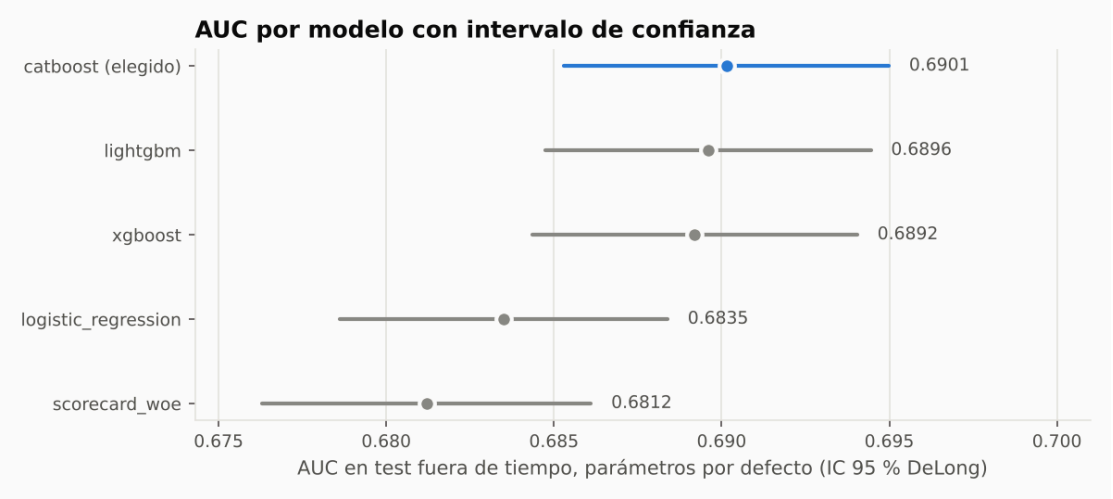
Selección Estadística en la Validación (DeLong con Ajuste de Holm)

Modelo	AUC val.	Δ vs. mejor	*p* Holm	Complejidad	Retenido	Elegido
CatBoost	0.6932	0.0000	1.000	2	Sí	**Sí**
LightGBM	0.6925	0.0007	.243	2	Sí	No
XGBoost	0.6917	0.0016	.011	2	Sí	No
Regresión logística	0.6860	0.0073	< .001	1	No	No

Nota. Un candidato se descarta solo si p Holm < .05 y ΔAUC ≥ 0.005. Complejidad: 1 = lineal, 2 = gradient boosting.

Figura 6

AUC por Modelo con Intervalo de Confianza del 95 %



Nota. Test fuera de tiempo con parámetros por defecto e intervalos de DeLong. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

P30. ¿Cuánto aportó el tuning con Optuna?

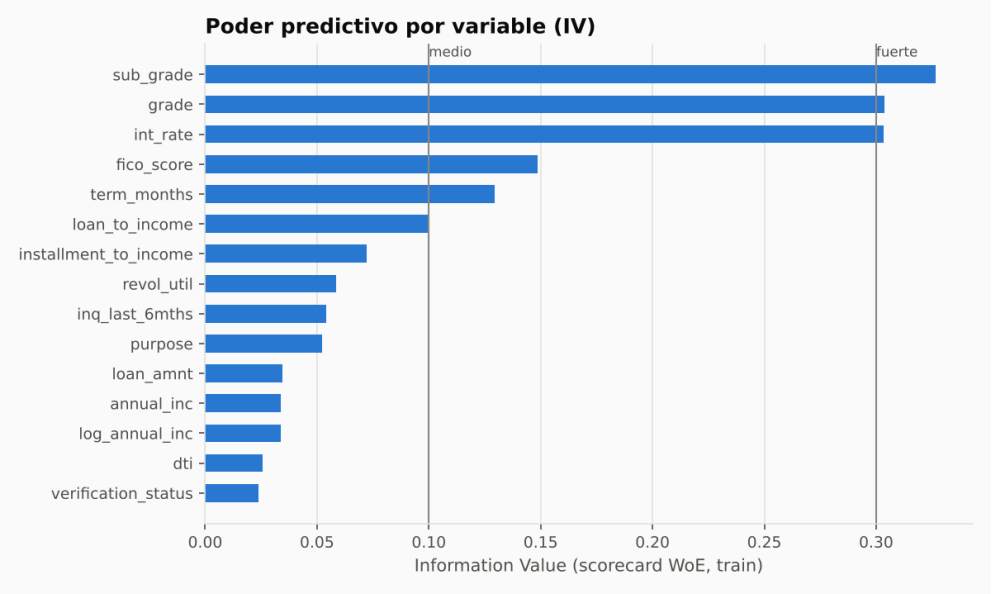
Poco. En la validación, donde se decide, mejoró 0.0003 de AUC (p = .433); en el test, 0.0010 (p = .002), una diferencia significativa pero prácticamente irrelevante. Lo interpreto como evidencia de que el techo lo pone la información disponible y no los hiperparámetros.

P31. ¿El modelo depende del scoring interno de Lending Club?

Lo comprobé con una ablación. Las variables `grade`, `sub_grade` e `int_rate` son salidas del scoring de Lending Club; se conocen al aprobar, así que no son leakage temporal, pero quería medir mi aporte propio. Sin ellas, el AUC baja de 0.6912 a 0.6815: una pérdida de 0.0097, IC 95 % [0.0077, 0.0117], p < .001. Aun así, el modelo conserva el 94.9 % de su poder discriminante (Gini) solo con variables del solicitante. Su Information Value está entre 0.30 y 0.33 (Figura 7): fuerte, pero por debajo del umbral de 0.5 que suele marcar sospecha de fuga.

Figura 7

Information Value por Variable



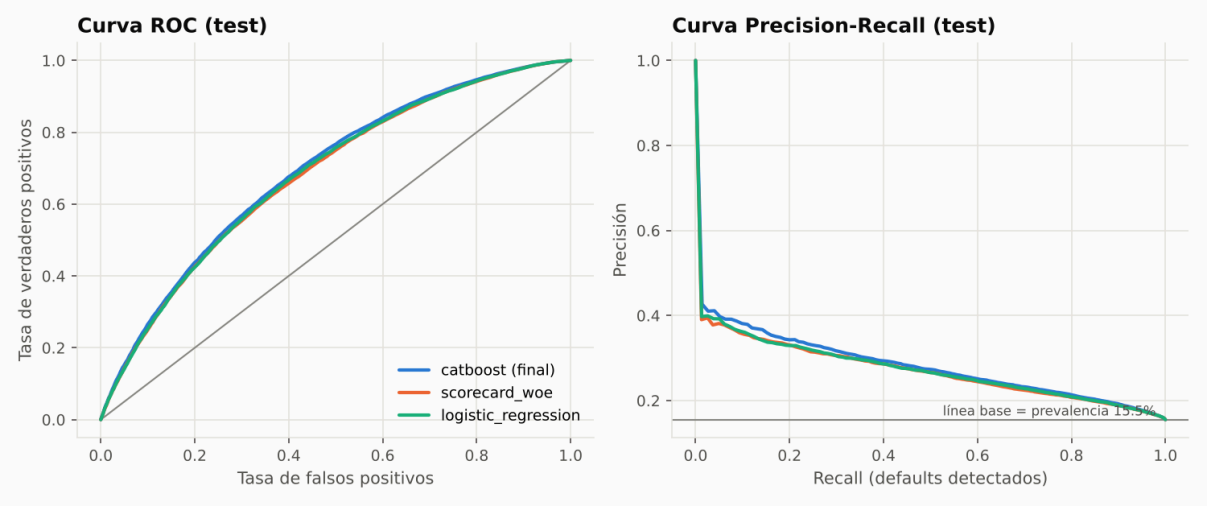
Nota. Valores de IV entre 0.1 y 0.3 se consideran de poder medio y mayores a 0.3, fuertes (Siddiqi, 2017). Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

P32. ¿Qué tan bien ordena la cartera?

El modelo alcanza un KS de 0.277 y un Gini de 0.382, con un PR-AUC de 0.279 frente a una prevalencia de 15.5 % (Figura 8). Por deciles (Figura 9), el 10 % más riesgoso tiene una tasa de default de 33.8 % y el 10 % más seguro, de 3.7 %; solo el primer decil concentra el 21.8 % de todos los defaults, con un lift de 2.18.

Figura 8

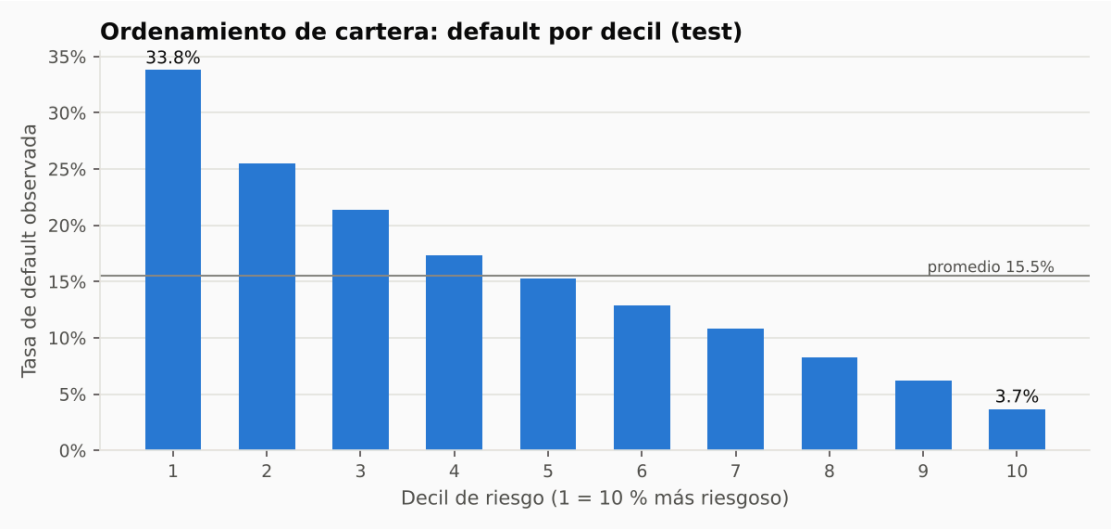
Curvas ROC y Precision-Recall en el Test



Nota. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

Figura 9

Tasa de Default por Decil de Riesgo



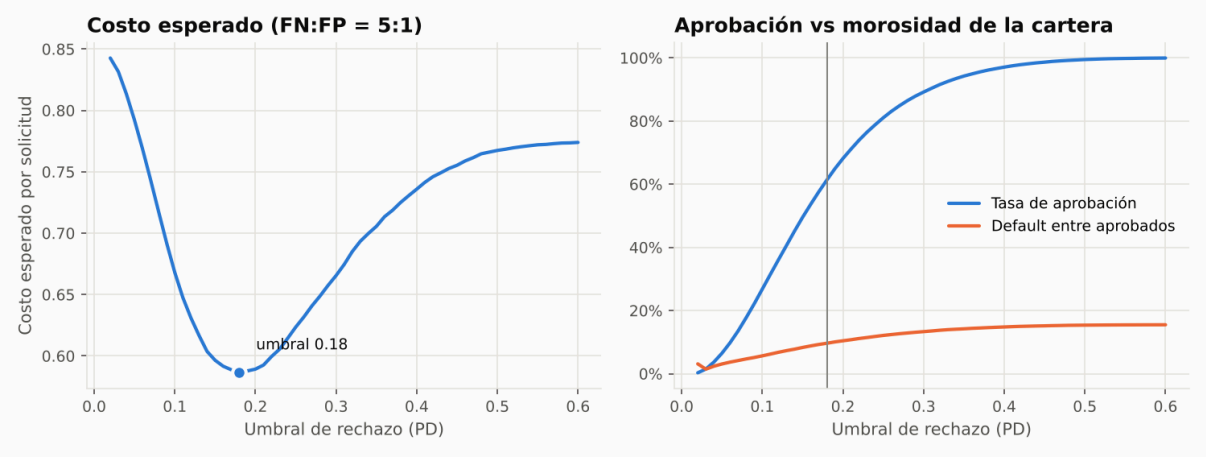
Nota. Decil 1 = 10 % de solicitudes con mayor PD. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

P33. ¿Qué impacto de negocio tiene la decisión?

Con el umbral de 0.18, el modelo aprueba el 61.3 % de las solicitudes, la morosidad entre los aprobados baja de 15.5 % a 9.6 % y se detecta el 61.9 % de los defaults (Figura 10 y Tabla 6). La tabla muestra el trade-off: un umbral más estricto, de 0.10, reduce la morosidad a 5.6 % pero aprueba solo al 26.8 %; uno más laxo, de 0.25, aprueba al 81.0 % con una morosidad de 12.1 %. El umbral elegido es el de menor costo esperado, y moverlo es una decisión de apetito de riesgo, no técnica.

Figura 10

Costo Esperado y Trade-off entre Aprobación y Morosidad



Nota. Costo con FN:FP = 5:1; la línea vertical marca el umbral elegido de 0.18. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

Tabla 6

Desempeño en el Test según el Umbral de Decisión

Umbral	Aprobación	Default entre aprobados	Recall	Precisión	Costo esperado
0.10	26.8 %	5.6 %	90.3 %	19.1 %	0.668
0.15	49.5 %	8.3 %	73.5 %	22.6 %	0.596
0.18 (elegido)	**61.3 %**	**9.6 %**	**61.9 %**	**24.8 %**	**0.586**
0.20	68.1 %	10.4 %	54.3 %	26.4 %	0.589
0.25	81.0 %	12.1 %	36.7 %	30.0 %	0.624
0.30	89.2 %	13.3 %	23.5 %	33.6 %	0.665

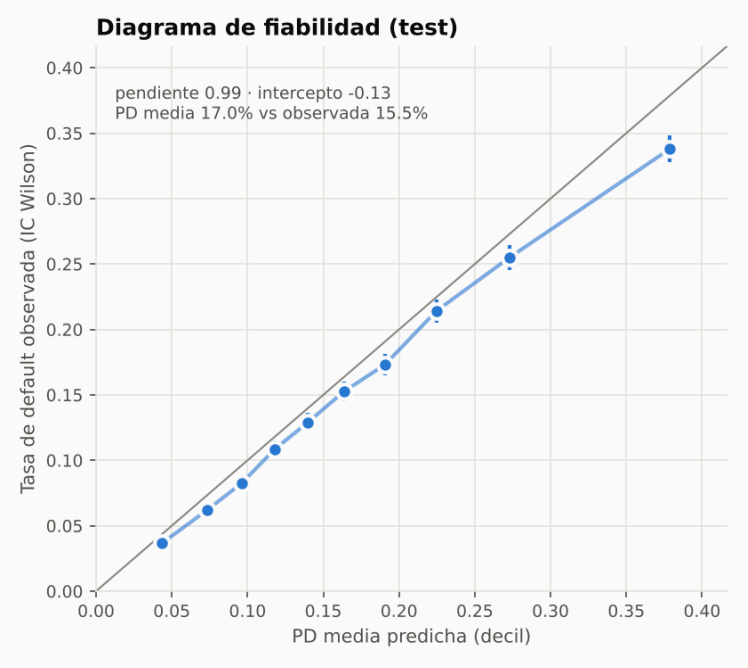
Nota. El umbral se eligió en la validación minimizando 5·FN + 1·FP y se mide aquí en el test. Recall y precisión se refieren a la detección de defaults.

P34. ¿Está bien calibrado el modelo?

Ordena bien, pero sobreestima el nivel. La pendiente de calibración es 0.99, casi ideal, así que la escala relativa es correcta. Sin embargo, el intercepto es −0.13 y la prueba de Spiegelhalter (1986) rechaza la calibración perfecta ($z = -10.45$, $p < .001$): la PD media es 17.0 % frente a 15.5 % observado (Figura 11). El Brier de 0.1233 mejora la referencia sin modelo de 0.1310 y el ECE es 0.015. Para aprobar o rechazar basta el ordenamiento; para pricing o pérdida esperada habría que recalibrar el intercepto con la validación.

Figura 11

Diagrama de Fiabilidad del Modelo Final



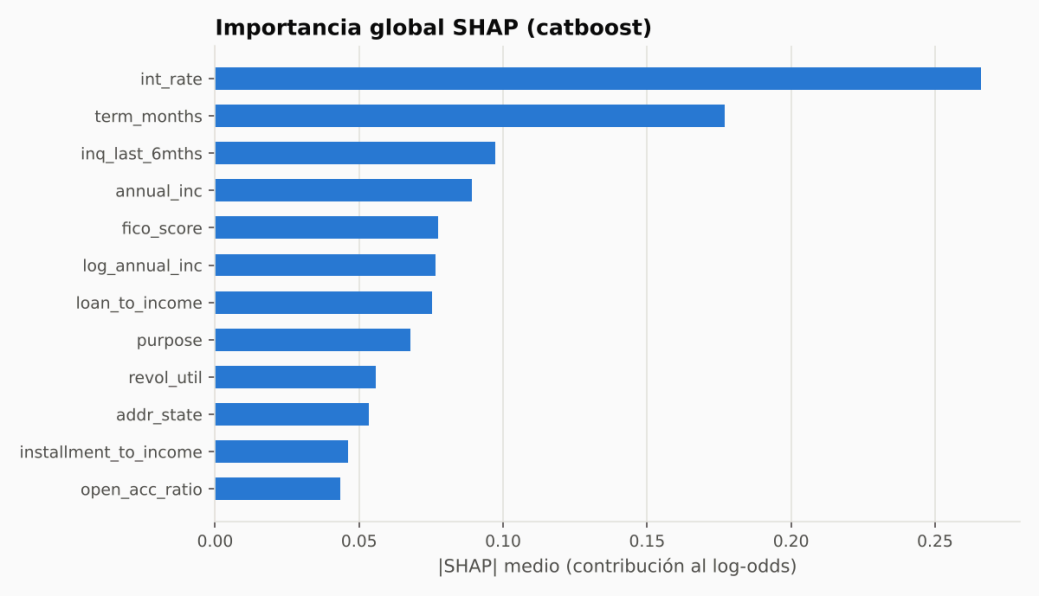
Nota. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

P35. ¿Qué variables pesan más en las predicciones?

La tasa de interés, el plazo, las consultas de crédito de los últimos seis meses, el ingreso anual y el FICO (Figura 12). Las direcciones tienen sentido económico: más tasa, más plazo, más consultas, más deuda relativa al ingreso y más utilización revolvente aumentan el riesgo; más ingreso y un mejor FICO lo reducen. Siempre aclaro que SHAP describe asociaciones que aprendió el modelo y no efectos causales.

Figura 12

Importancia Global SHAP



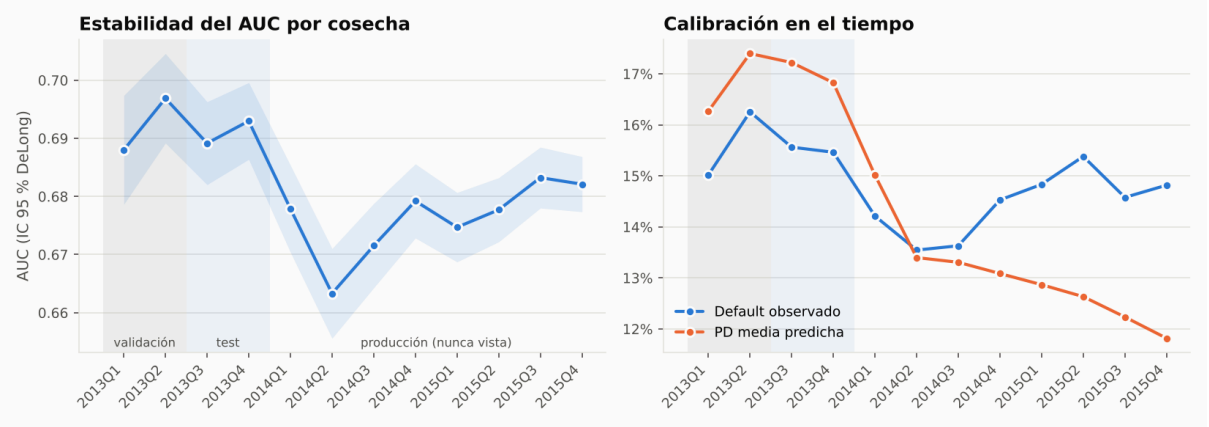
Nota. Media del valor absoluto de SHAP en la escala de log-odds; describe asociaciones aprendidas, no efectos causales. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

P36. ¿Es estable en el tiempo?

El AUC por trimestre se mueve entre 0.663 y 0.697 en doce cosechas, incluida la producción nunca vista (Figura 13). La discriminación es estable; lo que deriva es el nivel: en 2015Q4 la PD media es 11.8 % frente a 14.8 % observado. Esa deriva de calibración es precisamente la señal que el monitoreo usa para disparar reentrenamientos.

Figura 13

Estabilidad del AUC y de la Calibración por Cosecha



Nota. Las cosechas de 2014 y 2015 corresponden a producción nunca vista por el modelo. Generada por `scripts/build_report.py` sobre los préstamos emitidos entre 2007 y 2015 (Espinoza Luna, 2026).

Producción Simulada

P37. ¿Qué pasó cuando el modelo enfrentó datos que nunca había visto?

Corrí el replay en Databricks con el champion desplegado, un LightGBM (versión 3, AUC de test 0.6907, umbral 0.18). La Tabla 7 y la Figura 14 resumen el monitoreo. En diciembre de 2014, Page-Hinkley detectó concept drift: el AUC cayó 0.026 por debajo de la referencia. Esa alerta disparó automáticamente el reentrenamiento y generó el challenger v4, que pasó la validación offline y entró al A/B. Después, el AUC del champion se recuperó solo (0.677 y 0.693), lo que indica que parte de la caída fue variación entre cosechas y no un deterioro permanente.

Hubo dos hallazgos que considero los más valiosos. Primero, la calibración se degradó aunque el AUC se recuperara: la PD media bajó de 13.8 % a 12.8 % mientras el default observado subió de 13.9 % a 15.2 %. Segundo, el drift más fuerte no vino del cliente sino de la política comercial: `initial_list_status` tuvo un PSI de 3.0 porque en 2015 Lending Club pasó a vender casi todos sus préstamos como *whole loans*; le siguieron `purpose` (0.34) y `dti` (0.25).

Tabla 7

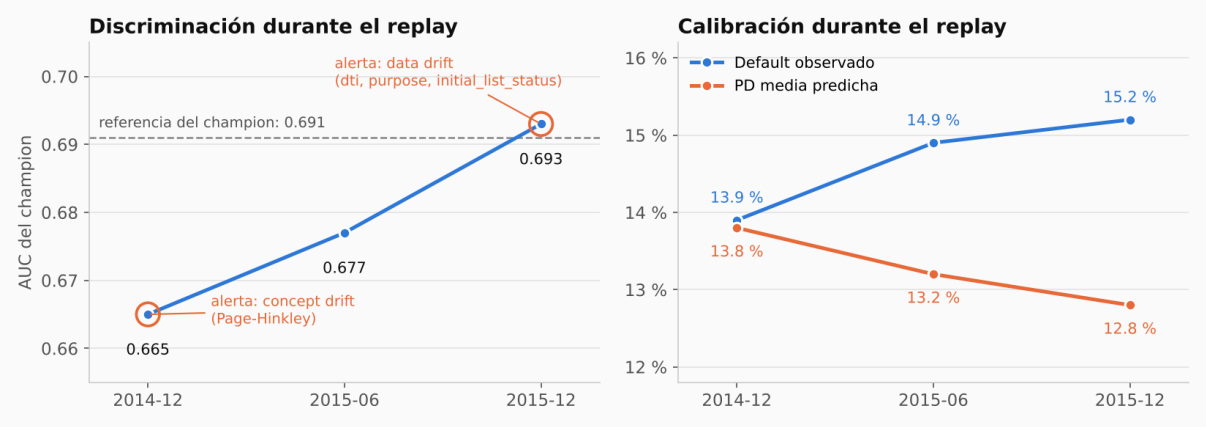
Monitoreo del Champion durante el Replay de 2014-2015

Reloj	Préstamos	Con desenlace	AUC	Default observado	PD media	Severidad	Motivo
2014-06	12,000	0	-	-	-	Estable	Solo data drift; aún no hay desenlaces
2014-12	12,000	8,191	0.665	13.9 %	13.8 %	**Alerta**	Concept drift (Page-Hinkley): AUC 0.026 bajo la referencia
2015-06	9,558	8,030	0.677	14.9 %	13.2 %	Estable	
2015-12	9,715	6,411	0.693	15.2 %	12.8 %	**Alerta**	Data drift en `dti`, `purpose` e `initial_list_status`

Nota. Corrida en Databricks del 4 de octubre de 2026. Champion: LightGBM, versión 3. Las etiquetas llegan con seis meses de retraso.

Figura 14

Discriminación y Calibración del Champion durante el Replay



Nota. Elaboración propia con las cifras de la corrida en Databricks del 4 de octubre de 2026 (Tabla 7). La ventana de 2014-06 no tiene desenlaces y no se grafica.

P38. ¿Cuál fue el resultado del A/B?

El challenger recibió el 20 % del tráfico. Al cierre, con 12,905 préstamos con desenlace en el champion y 3,291 en el challenger, el AUC fue 0.6807 frente a 0.6768: una diferencia de −0.0039, IC 95 % [−0.031, 0.023], p = .61. La morosidad de los aprobados

tampoco difirió de forma significativa (11.2 % frente a 12.3 %, $p = .13$). El sistema detuvo el test, eliminó al challenger y devolvió el 100 % del tráfico al champion (Tabla 8). La lección es que no se cambia un modelo en producción sin evidencia de que el nuevo es mejor.

Tabla 8

A/B Testing Champion contra Challenger sobre el Replay

Reloj	*n* champion v3	*n* challenger v4	AUC champion	AUC challenger	Diferencia [IC 95 %]	Decisión
2015-06	0	0	-	-	-	Continuar: aún sin desenlaces
2015-12	12,905	3,291	0.6807	0.6768	−0.0039 [−0.031, 0.023], $p = .61$	**Detener**

Nota. n = préstamos con desenlace por variante. El test se detiene al cumplir seis meses sin diferencia significativa.

P39. ¿Por qué el modelo desplegado es LightGBM si el reporte elige CatBoost?

Son dos corridas distintas del mismo procedimiento: el reporte reproducible corre en local con el CSV completo y el pipeline de producción corre en Databricks. Ambos modelos están en el grupo retenido por empate práctico, con diferencias de AUC menores a 0.005, y la elección dentro de ese grupo depende del tuning, que no es determinista entre corridas. Es la regla funcionando como se diseñó: entre modelos equivalentes, cualquiera es válido. Como mejora, generaría el reporte desde el artefacto que MLflow guarda con cada champion, para que el README y la demo muestren siempre el mismo modelo.

Discusión

Decisiones y Trade-offs

P40. ¿Cuál fue la decisión técnica más importante?

El diseño de la validación. Un split temporal, la eliminación del leakage y el control del sesgo de censura valen más que cualquier algoritmo; sin ellos habría reportado un AUC inflado que se desplomaría en producción. Lo segundo fue separar quién decide (la validación) de quién reporta (el test) y usar inferencia estadística para no confundir ruido con mejora.

P41. Un AUC de 0.69, ¿no es bajo?

Para este problema, no. Con solo variables de la solicitud, el AUC en Lending Club ronda 0.70, y los cuatro algoritmos quedaron a menos de 0.007 entre sí, lo que indica un techo de información. Lo que importa es el valor para el negocio: con ese AUC la morosidad de la cartera aprobada baja de 15.5 % a 9.6 %. Para subir el techo necesitaría más información, como datos de buró o de comportamiento de pago, y no más hiperparámetros.

P42. ¿Por qué no probaste un modelo más complejo, como una red neuronal?

La evidencia del propio proyecto apunta a que el límite es la información: el tuning aportó 0.0003 de AUC en la validación y cuatro algoritmos distintos empataron en la práctica. Además, la regla de parsimonia exige que la complejidad adicional gane de forma significativa y material. Un modelo más complejo tendría que superar esa vara y, a la vez, mantener la latencia por debajo de 5 ms y la explicabilidad por solicitud que exige un caso regulado.

Limitaciones

P43. ¿Cuáles son las principales limitaciones del proyecto?

Las reconozco de forma explícita, porque un revisor exigente las señalaría:

1. **Disponibilidad de la etiqueta.** El target de default en toda la vida del préstamo usa desenlaces que no se conocían al entrenar: un préstamo de 2012 termina en 2015 o 2017. El backtest respeta el orden de emisión, pero no la disponibilidad de la etiqueta, así que es algo optimista.
2. **Sesgo de selección.** El modelo solo aprende de préstamos aprobados y nunca vio a los rechazados (*reject inference*); el A/B también observa solo desenlaces de aprobados.
3. **Calibración de nivel.** La PD sobreestima la tasa observada en el test (17.0 % frente a 15.5 %) y la subestima en producción (12.8 % frente a 15.2 % en 2015-12).
4. **Potencia del A/B.** Con 3,291 préstamos con desenlace en el challenger, el IC de la diferencia de AUC es de ±0.027; solo podía detectar diferencias grandes. Detener significa sin evidencia, no que los modelos sean iguales.

5. **Falsas alarmas de drift.** El PSI alerta igual por cambios de política comercial que por cambios del cliente, y con muestras grandes casi cualquier diferencia es significativa.
6. **Simplificaciones.** El replay asume que el desenlace se conoce a los seis meses; no se evaluó la equidad por grupos; la Free Edition limita el compute y la demo pública es una foto del estado de producción.

P44. Con más tiempo, ¿qué mejorarías primero?

Primero, recalibraría el intercepto, o aplicaría Platt o regresión isotónica, con la validación, y monitorearía la calibración por cosecha con su propio umbral de alerta, porque es la falla que más afecta el uso de la PD en pricing y provisiones. Segundo, cambiaría el gate relativo champion-challenger, hoy una diferencia puntual de 0.002, por una prueba de DeLong coherente con la regla de selección. Tercero, redefiniría el target como PD a 12 meses, estándar de Basilea, o usaría un modelo de supervivencia en tiempo discreto que aproveche los préstamos censurados; eso también permitiría extender el replay a 2016-2018.

Después vendrían un análisis de potencia y pruebas secuenciales para el A/B, la corrección de Benjamini-Hochberg en el monitoreo de drift, la separación entre variables de política y de cliente, un experimento sobre la ventana de entrenamiento y un análisis de equidad.

Preguntas Difíciles

P45. Si el A/B no encontró diferencias, ¿no fue tiempo perdido?

Al contrario: evitó un cambio sin evidencia, que es lo que exigiría un comité de riesgo. Lo que sí reconozco es que el test tuvo poca potencia. Antes de lanzar otro, haría un análisis de tamaño de muestra para detectar una diferencia de 0.005 de AUC y ajustaría el tráfico o la duración del test.

P46. ¿Cómo responderías si un regulador pide explicar un rechazo?

Con tres capas. Para el caso individual, las tres variables con mayor contribución SHAP y la banda de riesgo. Para el modelo, el scorecard WoE como referencia interpretable en puntos y la tabla de Information Value. Para el proceso, la trazabilidad: el commit, la

versión Delta de los datos y los gates que pasó el modelo. Agregaría el análisis de equidad, que hoy es una limitación declarada.

P47. ¿Cómo llevarías esto a un banco real?

Mantendría el diseño y cambiaría la infraestructura: un workspace pagado con clusters dedicados por entorno, Online Tables para features en tiempo real y un service principal con OAuth en lugar de un token personal. En datos, agregaría información de buró y de comportamiento de pago; en gobierno, una validación independiente del modelo y el monitoreo de la calibración como métrica de primer nivel.

P48. ¿Qué aprendiste con este proyecto?

Aprendí que en riesgo crediticio el diseño de la validación pesa más que el algoritmo: un split temporal, sin leakage y sin sesgo de censura, cambia por completo qué tan creíble es un resultado. Aprendí a usar la inferencia estadística, con DeLong, Holm y márgenes prácticos, para decidir entre modelos en lugar de elegir el número más alto. Y aprendí que MLOps es sobre todo confiabilidad: reintentar solo lo idempotente, versionar datos y código juntos y no promover un modelo sin evidencia.

Referencias

Akiba, T., Sano, S., Yanase, T., Ohta, T. y Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631). ACM. https://doi.org/10.1145/3292500.3330701

Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785

Databricks. (s. f.). *Databricks Free Edition limitations*. https://docs.databricks.com/aws/en/getting-started/free-edition-limitations

DeLong, E. R., DeLong, D. M. y Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics, 44*(3), 837–845. https://doi.org/10.2307/2531595

Espinoza Luna, W. (2026). *Credit risk platform 2.0: Lending Club on Databricks* [Software]. GitHub. https://github.com/wilder14-eslu/credit-risk-platform-2.0

Gama, J., Medas, P., Castillo, G. y Rodrigues, P. (2004). Learning with drift detection. En A. L. C. Bazzan y S. Labidi (Eds.), *Advances in artificial intelligence: SBIA 2004* (pp. 286–295). Springer. https://doi.org/10.1007/978-3-540-28645-5_29

Google Cloud. (s. f.). *MLOps: Continuous delivery and automation pipelines in machine learning*. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. y Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. En *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates.

Lundberg, S. M. y Lee, S.-I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1/2), 100–115. https://doi.org/10.2307/2333009

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. y Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. En *Advances in Neural Information Processing Systems 31* (pp. 6638–6648). Curran Associates.

Siddiqi, N. (2017). *Intelligent credit scoring: Building and implementing better credit risk scorecards* (2.ª ed.). Wiley.

Sinha, D. (2023). *Practical machine learning on Databricks*. Packt Publishing.

Spiegelhalter, D. J. (1986). Probabilistic prediction in patient management and clinical trials. *Statistics in Medicine, 5*(5), 421–433. https://doi.org/10.1002/sim.4780050506

Sun, X. y Xu, W. (2014). Fast implementation of DeLong's algorithm for comparing the areas under correlated receiver operating characteristic curves. *IEEE Signal Processing Letters, 21*(11), 1389–1393. https://doi.org/10.1109/LSP.2014.2337313

wordsforthewise. (s. f.). *Lending Club loan data* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/wordsforthewise/lending-club

Apéndice A

Cifras Clave para Memorizar

La Tabla A1 reúne los números que más se preguntan en una entrevista sobre este proyecto. Conviene decirlos con su contexto: qué periodo, qué modelo y contra qué referencia.

Tabla A1

Cifras Clave del Proyecto

Indicador	Valor
Préstamos en el archivo	2,260,701 préstamos y 151 columnas (2007-2018)
Préstamos en el análisis estadístico	887,429 emitidos entre 2007 y 2015
Variables	26 de la solicitud + 5 derivadas; 22 columnas de leakage eliminadas
Split temporal	Entrenamiento 95,902 · validación 53,374 · test 81,430
Prevalencia de default en el test	15.5 %
AUC en el test (CatBoost + Optuna)	0.6912, IC 95 % [0.6863, 0.6960]
Gini · KS · PR-AUC	0.382 · 0.277 · 0.279
Rango de AUC entre los cuatro algoritmos	0.007 en el test
Margen práctico de la regla de selección	ΔAUC = 0.005 con *p* Holm < .05
Ventaja sobre regresión logística y scorecard WoE	+0.0066 y +0.0089 de AUC (ambas *p* Holm < .001)
Umbral de decisión	0.18, costo FN:FP = 5:1
Aprobación · morosidad de aprobados · recall	61.3 % · 9.6 % (vs. 15.5 % sin modelo) · 61.9 %
Primer decil de riesgo	Default 33.8 % (vs. 3.7 % en el último); 21.8 % de los defaults; lift 2.18
Calibración	Pendiente 0.99, intercepto −0.13; PD media 17.0 % vs. 15.5 % observado
Ablación sin `grade`, `sub_grade`, `int_rate`	AUC 0.6815 (−0.0097); conserva el 94.9 % del Gini
Champion desplegado en Databricks	LightGBM v3, AUC de test 0.6907
Concept drift detectado	2014-12, Page-Hinkley, AUC 0.026 bajo la referencia
Mayor PSI en producción	`initial_list_status` = 3.0 (cambio de política comercial)
Resultado del A/B	ΔAUC −0.0039, IC 95 % [−0.031, 0.023], *p* = .61: detener
Calidad de software	100 tests, cobertura ~94 %, CI en Python 3.11 y 3.12

Apéndice B

Preguntas Rápidas de Seguimiento

Los entrevistadores suelen cerrar con preguntas cortas de concepto. La Tabla B1 propone respuestas de una o dos frases.

Tabla B1

Preguntas de Concepto y Respuestas Breves

Pregunta	Respuesta breve
¿Qué es el PSI?	Compara la distribución de una variable entre la referencia y producción; un valor de 0.25 o más indica un cambio fuerte.
¿Qué mide el KS?	La máxima separación entre las distribuciones acumuladas del score de buenos y malos pagadores.
¿Cómo se relacionan Gini y AUC?	Gini = 2 × AUC − 1; con AUC 0.6912, Gini = 0.382.
¿Para qué sirve el test de DeLong?	Para comparar AUC correlacionados, medidos sobre los mismos préstamos, con un *p*-valor y un intervalo de confianza.
¿Por qué Holm y no Bonferroni?	Ambos controlan el error de familia en comparaciones múltiples, pero Holm tiene más potencia.
¿Qué son WoE e IV?	WoE es el logaritmo de la razón entre la proporción de buenos y de malos en un bin; IV resume el poder predictivo de la variable.
¿DDM o Page-Hinkley?	DDM vigila la tasa de error en línea; Page-Hinkley detecta un cambio sostenido en la media de una serie, aquí la log-loss.
¿Qué es el training-serving skew?	Una diferencia entre las transformaciones de entrenamiento y de servicio; lo evito serializando el preprocesador dentro del modelo.
¿Qué es el time travel de Delta?	Consultar una versión anterior de una tabla (`VERSION AS OF`) para reproducir los datos de un modelo.
¿Por qué asignar el A/B por hash?	Porque es determinista: el mismo solicitante siempre ve la misma variante y el experimento es reproducible.
¿Qué es un alias en MLflow?	Un puntero mutable a una versión del modelo, como `@champion`, que reemplaza a los antiguos stages.
¿Qué es reject inference?	El problema de no observar el desenlace de los rechazados: el modelo solo aprende de quienes fueron aprobados.
¿Qué es la maldición del ganador?	Elegir el mejor de varios modelos sobre el mismo test sesga hacia arriba el desempeño reportado.
¿Para qué sirve el cooldown?	Impone tres meses mínimos entre reentrenamientos para no reaccionar a ruido.